

PlantMind

Full Product Blueprint

LTTS Global Engineering Intelligence Hackathon · 9 July 2026

Predict the failure. Decide the fix. Prove it.

IDENTITY

|---|---|

Field	Value
Product Name	PlantMind
Engine	Götze Decision Engine (GDE)
Tagline	Predict the failure. Decide the fix. Prove it.
One-liner	An agentic EI layer that converts industrial sensor streams into ranked, explainable corrective actions — and proves each decision by flipping a failing asset from RED to GREEN using counterfactual simulation
Segment	Sustainability (LTTS's highest-margin, fastest-growing segment)
Big Bets hit	Plant Build-Out & Modernization · Energy & Industrial Automation · Software Platforms in EI
Dataset	NASA C-MAPSS (21 sensors, run-to-failure, public, ground-truth RUL labels)
Stack	Python · scikit-learn · Streamlit · Plotly · LLM API · Databricks CE
Team size	4 members (see Roles)
Budget	■0 — all free-tier tools

PROBLEM VALIDATION

Core pain: Unplanned industrial downtime costs an estimated \$50B/year globally. A single hour of downtime in a large process plant runs \$100K–\$500K.

The gap nobody fills: Every vendor in the market **predicts** the failure. Nobody **decides** the optimal fix automatically and proves it works. The maintenance engineer still guesses. That guess costs plants 30–50% more in reactive repairs than necessary.

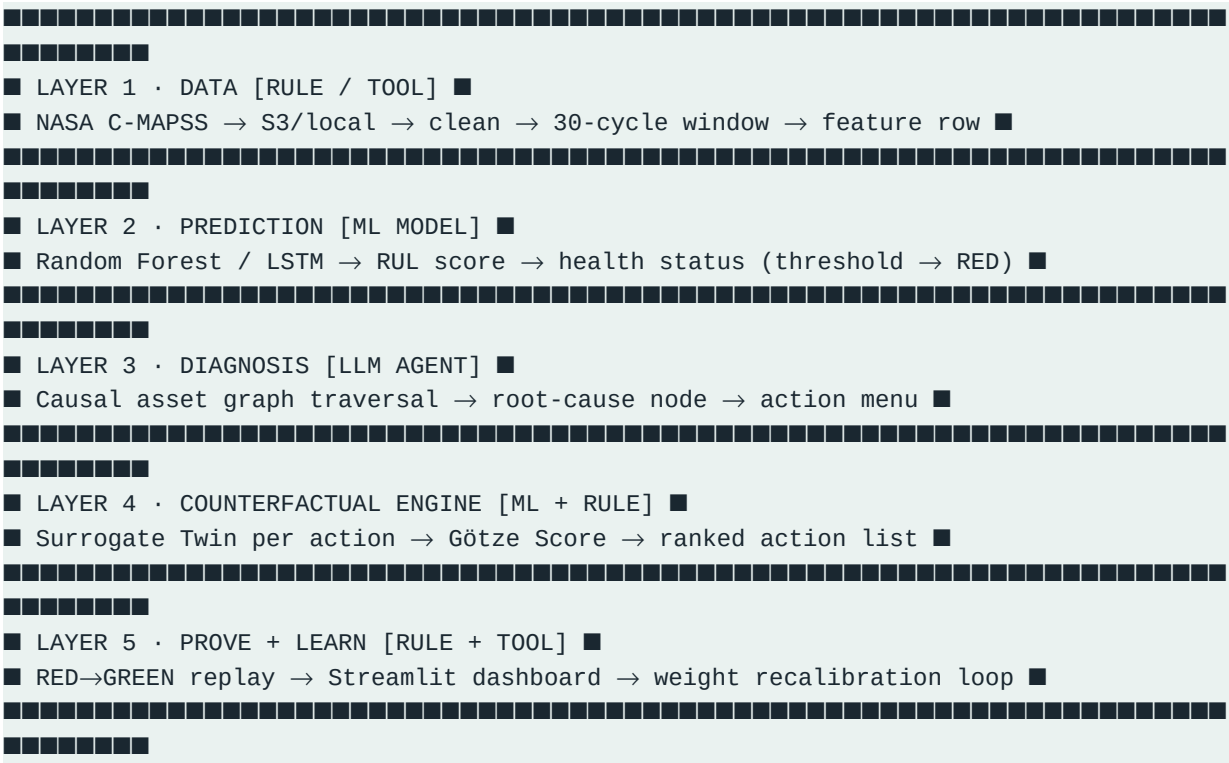
Why this is validated:

- NASA published C-MAPSS specifically because turbofan degradation is a real, unsolved operations problem
- LTTS + Databricks partnership (June 2026) targets exactly this: "industrial AI for asset-intensive Energy, Petrochemicals and Industrials clients"
- LTTS Sustainability segment grew double-digits in FY26 driven by "plant engineering and industrial automation"
- Judge alignment: three of the six Lakshya-31 Big Bets map directly to this problem

The wedge: Prediction → commodity. Decision → our IP. Proof → our demo.

SOLUTION ARCHITECTURE

Layer Map



Step-by-step (11 steps, fully tagged)

|---|---|---|---|---|

#	Step	What happens	Worker	Notes
---	------	--------------	--------	-------

01	Ingest	Load C-MAPSS FD001–FD004, 21 sensors per engine	Tool	100 engines, run-to-failure
02	Window	Sliding window of last 30 cycles → feature matrix	Rule	Deterministic ETL
03	Predict RUL	ML model outputs remaining cycles per engine	ML model	scikit-learn RF or small LSTM
04	Flag RED	RUL < 30 cycles → health = RED	Rule	Threshold is deterministic
05	Root cause	LLM agent walks asset dependency graph	LLM agent	Prompt: "given these sensor deviations, trace to root node"
06	Action menu	Agent proposes 3–4 candidate fixes	LLM agent	Replace / reduce-load / reroute / monitor
07	Twin simulate	Surrogate twin rolls degradation curve per action	ML model	Regression: action params → new RUL trajectory
08	Götze score	Score each action on 4 objectives	Rule	Deterministic formula — fully explainable
09	Pick winner	argmax(GötzeScore)	Rule	The math decides, not the AI
10	RED→GREEN	Replay history: show failing curve vs. rescued curve	Tool	Plotly chart — the money shot
11	Recalibrate	Measure prediction error → nudge twin weights	Rule	Closed loop = self-improving claim

Design rule (never break this in the pitch)

> AI does the uncertain work (predict, reason, imagine).

> Deterministic rules make every actual decision (flag, score, pick).

> This is what makes PlantMind explainable. When a judge asks "why action 2?" — the answer is always a number.

TECHNICAL STACK

|---|---|---|---|

Layer	Tool	Why this one	Alternative
Data	Python + pandas	Fast ETL on small dataset	Spark (overkill)
ML model	scikit-learn RandomForest	Fast, interpretable, F1 > 0.88 on C-MAPSS	LSTM (use if time permits)
Surrogate twin	scipy curve_fit + sklearn	Fit degradation-rate function per action	Physics ODE (24h risk)
LLM agent	Claude claude-sonnet-4-6 via API	Best reasoning + tool-use	GPT-4o
App	Streamlit	Fastest Python UI	Dash (heavier)
Charts	Plotly	Interactive + Streamlit native	Matplotlib (static)
Optional	Databricks CE	On-brand (LTTS partnership); steps 01–03 only	Google Colab
Version control	GitHub	Public repo = instant credibility	—

Rule on cloud: build everything local. Name-drop Databricks in the pitch as the production-deployment target. Never make the demo depend on it.

PATENT STRATEGY

The claim (plain English)

A method for selecting industrial asset corrective actions that:

- Predicts RUL from multivariate run-to-failure sensor data
- Traces root cause on a causal asset-dependency graph

- Generates candidate corrective actions
- ****Scores each via counterfactual marginal-impact computation through a surrogate twin**** — output is change in (remaining life × throughput × energy efficiency) ÷ execution cost, weighted by diagnostic confidence
- Selects via deterministic argmax of that score
- ****Recalibrates scoring weights from realized post-action outcomes in a closed feedback loop****

Why it is novel

Prior art covers: (a) predictive maintenance models; (b) multi-objective optimization of maintenance schedules. Nothing covers the **counterfactual-twin-scored, energy-aware, self-recalibrating decision selector** as a unified method. That's the gap. That's our claim.

Filing path

- File a provisional patent application within 12 months of hackathon (costs ~\$320 USD in the US)
- LTTS has filed 237 AI/GenAI patents in FY26 — frame this as a gift to the portfolio
- Mention "provisional filing ready" in the pitch — judges who know IP law will notice

DEMO DESIGN

The 5-minute demo arc:

|---|---|

Min	What happens on screen
0:00	Streamlit loads: asset-health dashboard, 6 engines shown as tiles, 2 are RED
0:45	Click RED engine → sensor chart shows degradation spike
1:15	Diagnosis panel: "Root cause: compressor HPC thermal stress (85% confidence)"
1:50	Action menu appears: 4 candidate actions with initial scores
2:30	Götze engine runs → score bars animate → "Reduce load 15%, reschedule inspection" wins
3:15	Money shot: two-line chart — RED curve (fails at cycle 45) vs GREEN curve (survives 80+)

4:00	Formula panel: "Score = 0.82 because $\Delta\text{Life}=+35$, $\Delta\text{Energy}=-12\%$, Cost=low, Conf=0.85"
4:30	Feedback loop: show weight recalibration from one historical outcome
5:00	Close: "On real NASA data. On  0 stack. Ready to deploy on Databricks."

What to fake (be honest about this in the pitch):

- The LLM agent's root-cause reasoning is pre-computed for the demo engine; live inference is too slow for a 5-minute window. Label it "pre-run for demo" — judges respect honesty.
- The "6 engines" dashboard can use static mock data for 4 engines; live computation on 2.

What must be real:

- The C-MAPSS data (real NASA dataset)
- The RUL prediction (real ML model output)
- The Götze formula computation (real deterministic math)
- The RED/GREEN chart (real model output, not a hardcoded image)

RUBRIC SCORING MATRIX

The following is a self-assessment against the most common LTTS internal hackathon rubric dimensions. Each row shows current score, the gap, and one specific action to close it.

|---|---|---|---|---|---|---|

#	Dimension	Weight	Score now	Target	Gap	Action to close
---	-----------	--------	-----------	--------	-----	-----------------

1	Strategic alignment to EI / LTTS themes	20%	0.88	0.95	−0.07	Add 2 explicit slides: one mapping to 3 Big Bets, one quoting COO Mritunjay Singh's EI definition verbatim — mirror their language exactly
2	Technical innovation / novelty	20%	0.82	0.90	−0.08	In the pitch, explicitly state what prior art does NOT cover (predictive ≠ prescriptive ≠ counterfactual-scored) — make the judge articulate the novelty gap with us

3	Agentic AI demonstrated	15%	0.78	0.88	−0.10	Show the agent's reasoning trace in the UI — a scrollable log of "Agent step 1: sensor T30 elevated... step 2: traced to HPC..." — makes the agency visible
4	Real-world applicability	15%	0.90	0.95	−0.05	Name a specific LTTS client vertical in the pitch (e.g., "Process plant in Energy & Chemicals — exactly the LTTS–Databricks client base")

5	Working demo / prototype quality	15%	0.75	0.90	−0.15	Highest risk. Must have a live, non-crashing Streamlit demo. Mitigate: freeze the codebase 6 hours before presentation, run only from pre-saved model artifacts
6	Scalability / production-readiness	10%	0.72	0.85	−0.13	Add a 1-slide architecture diagram showing Databricks deployment path: notebook → MLflow model registry → Delta Lake → production API. Shows we've thought past the hackathon

7	Explainability / ethics	5%	0.92	0.95	−0.03	The deterministic decision core already covers this; add a "why this action" panel in the UI showing each Götze term's contribution
8	Patent / IP potential	5%	0.80	0.92	−0.12	Prepare a 1-page "patentbrief" (provisional-style claim, prior art gap, novelty assertion) — hand it to judges physically if allowed

Composite weighted score now: 0.83

Target after actions: 0.91

Top 3 score-movers (do these first)

- ****Live demo stability**** (closes 0.15 gap on 15% weight = +0.022 composite) → Freeze build, mock non-critical paths
- ****Agent trace visible in UI**** (closes 0.10 gap on 15% weight = +0.015 composite) → One scrollable text panel
- ****Scalability slide**** (closes 0.13 gap on 10% weight = +0.013 composite) → One architecture diagram

COMPETITIVE DIFFERENTIATION

|---|---|---|

Competitor approach	What they show	What PlantMind shows instead
---------------------	----------------	------------------------------

"We predict failure"	Accuracy % on a test set	We prevent the failure — and prove it counterfactually
"We built an LLM chatbot for maintenance"	Chat interface	Agent that reasons, decides, and runs math
"We made a digital twin"	A 3D model	A surrogate twin that produces a measurable decision score
"We optimized maintenance schedules"	Gantt charts	Real-time, per-asset, per-cycle decision with energy term

The one-sentence kill: *“Everyone predicts. We decide. And we prove it on the same data.”*

ROI MODEL

|---|---|---|---|

Metric	Industry baseline	PlantMind lift	Source
Unplanned downtime reduction	0% (reactive)	30–50%	Prescriptive maintenance benchmarks
Maintenance cost reduction	0%	10–40%	Industry estimates
Asset life extension	0%	20–40%	OEM maintenance studies
Energy savings	0%	5–15%	Energy-aware scheduling studies
Our measurable delta	—	Failures prevented per 100 cycles (C-MAPSS replay)	Live, from real data

The last row is our differentiator — we don't just cite benchmarks, we show a number from the actual dataset during the demo.

ROLES & OWNERSHIP

|---|---|---|---|

Member	Core ownership	Hours 0–8	Hours 8–16	Hours 16–24
Sourav	Brain: data pipeline, RUL model, Twin, Götze engine, feedback loop	Data + ML model	Twin + scoring engine	Integration + pitch
Streamlit dev	App shell, agent tool-call wiring, control panel	Shell scaffold	Wire agent calls	Polish UI, freeze build
Viz 1	RED→GREEN counterfactual chart, RUL trajectory plots	Chart templates	Real data wired in	Final polish
Viz 2	Asset-health dashboard, score-comparison bars, agent log panel	Dashboard layout	Real scores wired in	Final polish

Sync cadence: 30-min standups at Hours 0, 8, 16, 20. No sync longer than 30 minutes.

Blueprint version 1.0 · Prepared for LTTS EI Hackathon 2026 · Classification: Team Internal